



What is require()?

Definition: `require()` is a built-in function in Node.js used to load and use modules (files or libraries) inside your application.

Key Points:

- Imports code from another file or package
- Executes the module once when first loaded
- Returns the exported content (`module.exports`)
- Enables modular, scalable code structure

Why It Matters:

- Avoids writing everything in one file
- Promotes code reuse
- Makes large applications manageable

Basic Example:

```
// Importing a local module
const math = require('./math');
```

One-line Understanding:

➔ **require()** = “Load this file/module and give me what it exports.”

Why require() Exists

The Core Problem:

- Hard to read
- Hard to debug
- Impossible to scale



👉 This is not
"development" —
this is chaos.

```
// One huge file doing everything
function add(a, b) { return a + b; }
function multiply(a, b) { return a * b; }
// hundreds of functions...
```

How require() Solves It:

```
// math.js

module.exports = {
  add: (a, b) => a + b
};
```

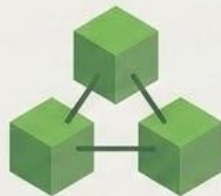
```
// app.js

const math = require('./math');
console.log(math.add(2, 3));
```



The Solution — Modularization

- Split code into separate files (modules)
- Each file handles one responsibility
- Reuse logic wherever needed



Key Benefits:

- Cleaner code structure
- Reusability across files
- Easier debugging and maintenance
- Scalable architecture

Reality Check: If you're not breaking your code into modules, you're not building real systems — you're writing beginner scripts.

How require() Works Internally

Stop treating require() like magic. It's a **process**, and if you don't understand this, debugging will destroy you.

Step-by-Step Execution:

1. **Path Resolution**
 - Finds the module (`../file`, `node\_modules`, or core module)
2. **Loading the File**
 - Reads the file from disk
3. **Wrapping the Module**
 - Node wraps it inside a function:
4. **Execution**
 - Runs the code once
5. **Return Value**
 - Returns `module.exports`

```
(function(exports, require, module, __filename, __dirname) {  
  // your code  
});
```

4. **Execution**
 - Runs the code once
 - Returns `module.exports`

Critical Concept — Caching (Most People Ignore This)

```
const a = require('./test');  
const b = require('./test');
```

- The file runs **only once**
 - Second call returns **cached result**
- 👉 This means:
- State can be shared
 - Bugs can persist if you mutate exports

Why This Matters in Real Systems:

- Improves performance (no repeated execution)
- Enables **shared state** between modules
- Can cause **hidden bugs** if misunderstood

Reality Check:

If you don't understand caching and execution flow, you'll:

- Write unpredictable code
- Fail in backend interviews
- Struggle with frameworks built on top of Node.js

This slide is where beginners separate from actual engineers.

Types of Modules in require()

If you don't know what kind of module you're loading, you're just guessing — and guessing breaks systems.



1. Core Modules (Built into Node.js)

- No installation needed
- Provided by Node itself

```
const fs = require('fs');  
const path = require('path');
```

➔ Used for file handling, paths, servers, etc.



2. Local Modules (Your Own Code)

- Files you create inside your project
- Must use relative paths (`.`, `../`)

```
const math = require('./math');
```

➔ If your path is wrong, your app crashes — simple.



3. Third-Party Modules (From npm)

- Installed using npm
- Stored in `node\_modules`

```
const express = require('express');
```

➔ Frameworks like Express.js come from here

How Node Decides What to Load

When you write: `require('something');`

1. Core modules
↓
2. `node\_modules` folder
↓
3. Fails → throws error

This is basic — but if you mess this up, nothing in backend development will work reliably.

Reality Check

- If you: Don't understand module types
- Mix up paths
- Don't know where your code is coming from

Then debugging will destroy your confidence.

module.exports vs exports

This is where most beginners mess up and then cry "Node.js is confusing." **It's not** — your understanding is weak.

What is Being Returned by require()?

➔ **require() always returns module.exports**
If this is wrong → your import is useless.

Way 1 — Using module.exports (Primary Method)

```
// file.js
module.exports = function () {
  console.log("Hello");
};
```

✓ Replaces the entire export

```
// app.js
const fn = require('./file');
fn();
```

✓ Clean and predictable

Way 2 — Using exports (Shortcut)

```
// file.js
exports.add = (a, b) => a + b;
exports.sub = (a, b) => a - b;
```

✓ Adds properties to module.exports

```
// app.js
const math = require('./file');
console.log(math.add(2, 3));
```

The Trap (Where People Fail)

```
// file.js
exports = function () {}; // ❌ WRONG
```

➔ This breaks the connection with module.exports
➔ require() will NOT return this

Correct Mental Model

- module.exports = actual object returned
- exports = reference (shortcut to that object)

Rule You Should Follow

👉 If you're not 100% clear, **always** use **module.exports**

Reality Check:

If you: • Don't understand this difference • Randomly mix both
You'll get undefined, waste hours debugging, and blame Node instead of your fundamentals.

This is not advanced — this is basic competency.

Common Mistakes with require()

This is where beginners expose themselves. These aren't "small errors" – they show you don't understand how modules work.

1. Wrong File Path

```
const data = require('data'); // ❌ wrong  
const data = require('./data'); // ✅ correct
```

- 👉 Forget `./` → Node looks in `node\_modules` instead of your file
- 👉 Result: **Module not found**

2. Forgetting module.exports

```
// file.js  
function test() {}  
  
const t = require('./file');  
console.log(t); // ❌ undefined
```

- 👉 No export = nothing returned

3. Misusing exports

```
exports = { a: 1 }; // ❌ wrong
```

- 👉 Breaks reference with `module.exports`
- 👉 Your import silently fails

4. Thinking require() Works in Browser

- 👉 It does **NOT** work in browsers
- 👉 It's specific to Node.js

Use ES Modules (`import/export`) in frontend instead.

Final Reality Check

- If you: Can't debug a "module not found" error
- Don't know what `require()` returns
- Break exports repeatedly

Then you don't understand backend – you're just assembling code blindly.

Fix this now, or it will block **everything**: frameworks, APIs, system design – all of it.

5. Ignoring Caching Behavior

```
const a = require('./config');  
const b = require('./config');
```

- 👉 Runs only once
- 👉 Returns cached version

⚠️ If you modify it:

```
a.value = 10;
```

→ `b.value` also changes

6. Poor Project Structure

- * Dumping everything in one folder
- * Random file naming
- * No separation of concerns

👉 **Result:**

- * Confusion
- * Bugs
- * No scalability